

# Gender dynamics experiments

Eduardo García-Portugués, Carlos Vaquero

2026-01-20

Running experiment 2 with subset Sop\_features\_non\_cat

## Load libraries

```
# Load libraries
library(knitr)
library(dplyr)
library(recipes)
library(rsample)
library(yardstick)
library(glmnet)
library(ggplot2)
library(caret)
library(tibble)
sessionInfo()
```

```
R version 4.5.2 (2025-10-31)
Platform: aarch64-apple-darwin20
Running under: macOS Sequoia 15.7
```

```
Matrix products: default
```

```
BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/
```

```
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK ve
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: Europe/Madrid
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods   base
```

other attached packages:

```
[1] tibble_3.3.1    caret_7.0-1      lattice_0.22-7   ggplot2_4.0.1  
[5] glmnet_4.1-10   Matrix_1.7-4     yardstick_1.3.2  rsample_1.3.1  
[9] recipes_1.3.1   dplyr_1.1.4      knitr_1.51
```

loaded via a namespace (and not attached):

```
[1] gtable_0.3.6      shape_1.4.6.1     xfun_0.55  
[4] vctrs_0.6.5       tools_4.5.2       generics_0.1.4  
[7] stats4_4.5.2      parallel_4.5.2    ModelMetrics_1.2.2.2  
[10] pkgconfig_2.0.3   data.table_1.18.0 RColorBrewer_1.1-3  
[13] S7_0.2.1          lifecycle_1.0.5   stringr_1.6.0  
[16] compiler_4.5.2    farver_2.1.2      codetools_0.2-20  
[19] htmltools_0.5.9   class_7.3-23      yaml_2.3.12  
[22] prodlim_2025.04.28 pillar_1.11.1     furrr_0.3.1  
[25] tidyr_1.3.2       MASS_7.3-65       gower_1.0.2  
[28] iterators_1.0.14  rpart_4.1.24      foreach_1.5.2  
[31] nlme_3.1-168      parallelly_1.46.1 lava_1.8.2  
[34] tidyselect_1.2.1  digest_0.6.39     stringi_1.8.7  
[37] future_1.68.0     reshape2_1.4.5    purrr_1.2.1  
[40] listenv_0.10.0    splines_4.5.2     fastmap_1.2.0  
[43] grid_4.5.2        cli_3.6.5         magrittr_2.0.4  
[46] survival_3.8-3    future.apply_1.20.1 withr_3.0.2  
[49] scales_1.4.0      lubridate_1.9.4    timechange_0.3.0  
[52] rmarkdown_2.30    globals_0.18.0    otel_0.2.0  
[55] nnet_7.3-20       timeDate_4051.111 evaluate_1.0.5  
[58] hardhat_1.4.2     rlang_1.1.7       Rcpp_1.1.1  
[61] glue_1.8.0        pROC_1.19.0.1     ipred_0.9-15  
[64] jsonlite_2.0.0    plyr_1.8.9        R6_2.6.1
```

## Read datasets

```
# Load csv  
df <- read.csv(paste0("Experiment_", experiment, ".csv"),  
               stringsAsFactors = TRUE)  
  
# Choose target column based on experiment  
target_col <- case_when(  
  experiment %in% c(1, 2) ~ "Gender",  
  experiment == 3 ~ "Singer_Sex"  
)  
  
# Rename target to 'y'  
df <- df |>  
  rename(y = !!target_col)  
  
# Number of observations per group
```

```
df |>
  group_by(y) |>
  summarise(n = n(), .groups = "drop")
```

```
# A tibble: 2 x 2
```

```
  y         n
  <fct> <int>
1 Female  568
2 Male    669
```

```
# Split into train / test (85% / 15%)
set.seed(123)
split_obj <- initial_split(df, prop = 0.85)
train <- training(split_obj)
test <- testing(split_obj)
```

```
# Determine subset based ONLY on training data
features_list_train <- get_features_subsets_voice(train)
if (!(subset_name %in% names(features_list_train))) {
  stop("Subset ", subset_name, " not found")
}

# Select features based on subset name
selected_features <- colnames(features_list_train[[subset_name]])
train <- train |>
  select(all_of(selected_features), y)
test <- test |>
  select(all_of(selected_features), y)
dim(train)
```

```
[1] 1051  87
```

```
dim(test)
```

```
[1] 186  87
```

```
# Combined dataset for recipes
combined <- bind_rows(train, test)
```

## Preprocessing recipes

```

# Preprocessing recipe
preprocess_recipe <- function(data) {
  recipe(y ~ ., data = data) |>
    # Preprocess all predictors BEFORE dummifying
    step_zv(all_predictors()) |>
    step_nzv(all_predictors()) |>
    step_filter_missing(all_predictors(), threshold = 0.05) |>
    step_impute_knn(all_predictors(), neighbors = 5) |>
    # Collapse rare factor levels (BEFORE dummy creation)
    step_other(all_nominal_predictors(), threshold = 0.10) |>
    # Preprocess numeric predictors only
    step_corr(all_numeric_predictors(), threshold = 0.80) |>
    step_lincomb(all_numeric_predictors(), max_steps = 20) |>
    step_normalize(all_numeric_predictors()) |>
    # Create dummy variables (AFTER normalization)
    # Use one-hot encoding such that no dummy variable is dropped and we can
    # later drop the "other" level manually (so it becomes the reference level
    # and we avoid hard-to-explain dummies in the final model)
    # step_dummy(all_nominal_predictors(), one_hot = TRUE) |>
    step_dummy(all_nominal_predictors(), one_hot = FALSE) |>
    # Drop the dummies for the "other" level
    step_rm(ends_with("_other")) |>
    # Post-dummy cleanup
    step_zv(all_predictors()) |>
    step_nzv(all_predictors()) |>
    step_lincomb(all_predictors(), max_steps = 20)
}

```

```

# Train recipe using train data
rec_train <- preprocess_recipe(data = train)
prep_rec_train <- prep(rec_train)

# Bake train data
train_bake <- bake(prep_rec_train, new_data = NULL)
test_train_bake <- bake(prep_rec_train, new_data = test)

# Train recipe using combined data (train + test)
rec_all <- preprocess_recipe(data = combined)
prep_all <- prep(rec_all)

# Fully baked dataset for the final model fit
all_bake <- bake(prep_all, new_data = NULL)

# Export datasets for future reference and use
dir.create("preprocessed_datasets", showWarnings = FALSE)
write.csv(train_bake, paste0("preprocessed_datasets/train_bake_exp", experiment,
                             "_sub_", subset_name, ".csv"),
          row.names = FALSE)

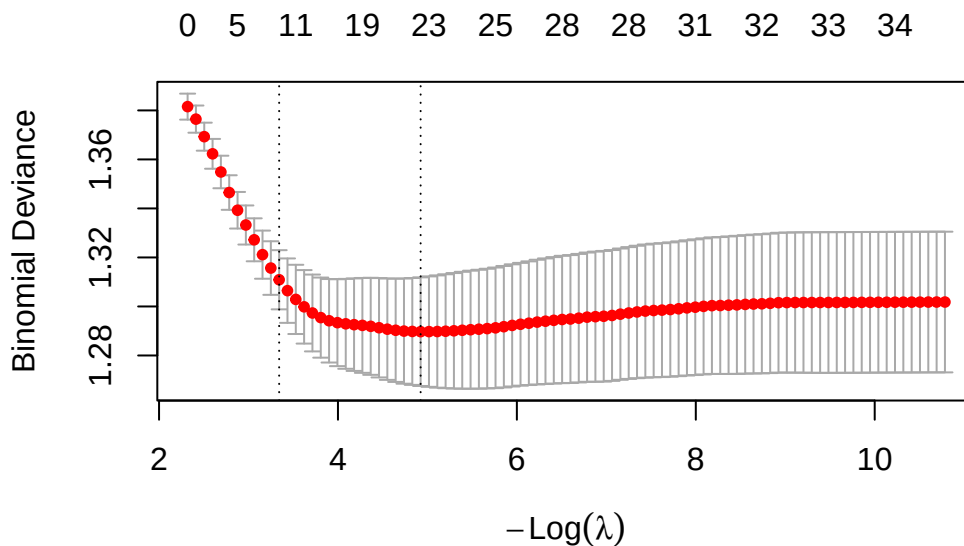
```

```
write.csv(test_train_bake, paste0("preprocessed_datasets/test_train_bake_exp",
                                experiment, "_sub_", subset_name, ".csv"),
          row.names = FALSE)
write.csv(all_bake, paste0("preprocessed_datasets/all_bake_", experiment,
                           "_sub_", subset_name, ".csv"),
          row.names = FALSE)
```

## Lasso logistic

### Fitting

```
# Fit by 10-fold cross-validation
set.seed(42)
train_bake_no_y <- train_bake |>
  select(-y) |>
  as.matrix()
cv1_train <- cv.glmnet(x = train_bake_no_y, y = train_bake$y,
                      family = "binomial", alpha = 1, nfolds = 10)
plot(cv1_train)
```



### Predictions

```
# Predictions
test_train_bake_no_y <- test_train_bake |>
  select(-y) |>
```

```

as.matrix()
y_test_hat_LL <- predict(cv1_train, newx = test_train_bake_no_y,
                        s = "lambda.1se", type = "class")
p_test_hat_LL <- predict(cv1_train, newx = test_train_bake_no_y,
                        s = "lambda.1se", type = "response")

```

## Metrics

```

# Metrics
acc_LL <- mean(test$y == y_test_hat_LL)
auc_LL <- auc_binary(test$y, p_test_hat_LL)
f1_LL <- f1(test$y, y_test_hat_LL)
data.frame("Accuracy" = acc_LL, "AUC" = auc_LL, "Macro_F1" = f1_LL) |>
  kable(digits = 4)

```

| Accuracy | AUC    | Macro_F1 |
|----------|--------|----------|
| 0.6774   | 0.7111 | 0.671    |

## Relevant coefficients

```

# Extract nonzero coefficients for lambda.1se and order them by absolute value
coefs1_train <- predict(cv1_train, s = "lambda.1se", type = "coefficients") |>
  as.matrix() |>
  as.data.frame() |>
  rownames_to_column(var = "feature") |>
  rename("coefficient" = "lambda.1se")
coefs1_train |>
  filter(coefficient != 0) |>
  arrange(desc(abs(coefficient))) |>
  kable(digits = 4)

```

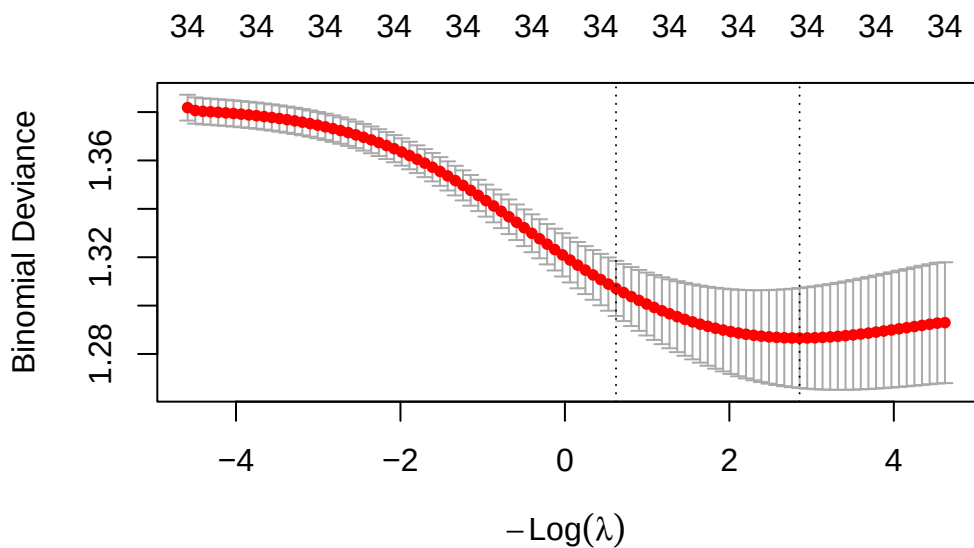
| feature                              | coefficient |
|--------------------------------------|-------------|
| PartSop_HighestNoteIndex             | -0.2290     |
| (Intercept)                          | 0.1670      |
| PartSop_Ambitus                      | 0.1212      |
| PartSop_DescendingIntervallicStd     | 0.1204      |
| PartSop_LargestSemitonesAsc          | 0.1096      |
| PartSop_IntervalsBeyondOctaveAsc_Per | 0.0725      |
| PartSop_LargestSemitonesDesc         | -0.0510     |
| PartSop_AscendingIntervallicMean     | 0.0262      |
| PartSop_AbsoluteIntervallicTrimRatio | 0.0200      |

| feature                             | coefficient |
|-------------------------------------|-------------|
| PartSop_VoicePresence               | 0.0190      |
| PartSop_IntervalsMinorAll_Per       | -0.0187     |
| PartSop_IntervalsDiminishedDesc_Per | -0.0014     |

## Ridge logistic model

### Fitting

```
# Fit by 10-fold cross-validation
set.seed(42)
cv0_train <- cv.glmnet(x = train_bake_no_y, y = train_bake$y,
                      family = "binomial", alpha = 0, nfolds = 10)
plot(cv0_train)
```



### Predictions

```
# Predictions
y_test_hat_RL <- predict(cv0_train, newx = test_train_bake_no_y,
                        s = "lambda.1se", type = "class")
p_test_hat_RL <- predict(cv0_train, newx = test_train_bake_no_y,
                        s = "lambda.1se", type = "response")
```

## Metrics

```
# Metrics
acc_RL <- mean(test$y == y_test_hat_RL)
auc_RL <- auc_binary(test$y, p_test_hat_RL)
f1_RL <- f1(test$y, y_test_hat_RL)
data.frame("Accuracy" = acc_RL, "AUC" = auc_RL, "Macro_F1" = f1_RL) |>
  kable(digits = 4)
```

| Accuracy | AUC    | Macro_F1 |
|----------|--------|----------|
| 0.6452   | 0.6888 | 0.6329   |

## Relevant coefficients

```
# Extract coefficients and sort them by absolute value
coefs0_train <- predict(cv0_train, s = "lambda.1se", type = "coefficients") |>
  as.matrix() |>
  as.data.frame() |>
  rownames_to_column(var = "feature") |>
  rename("coefficient" = "lambda.1se")
coefs0_train |>
  filter(coefficient != 0) |>
  arrange(desc(abs(coefficient))) |>
  head(30) |>
  kable(digits = 4)
```

| feature                               | coefficient |
|---------------------------------------|-------------|
| (Intercept)                           | 0.1937      |
| PartSop_HighestNoteIndex              | -0.1035     |
| PartSop_LargestIntervalAll_P8         | -0.0645     |
| PartSop_DescendingIntervallicStd      | 0.0615      |
| PartSop_LargestSemitonesAsc           | 0.0615      |
| PartSop_IntervalsBeyondOctaveAsc_Per  | 0.0600      |
| PartSop_AbsoluteIntervallicTrimRatio  | 0.0586      |
| PartSop_Ambitus                       | 0.0548      |
| PartSop_LargestSemitonesDesc          | -0.0548     |
| PartSop_AscendingIntervallicMean      | 0.0492      |
| PartSop_IntervalsDiminishedDesc_Per   | -0.0484     |
| PartSop_VoicePresence                 | 0.0461      |
| PartSop_IntervallicKurtosis           | 0.0458      |
| PartSop_IntervalsMinorAll_Per         | -0.0410     |
| PartSop_TrimmedAbsoluteIntervallicStd | 0.0374      |



| feature                               | coefficient |
|---------------------------------------|-------------|
| PartSop_IntervalsBeyondOctaveDesc_Per | 0.0342      |
| PartSop_AverageDuration               | 0.0273      |
| PartSop_DottedRhythm                  | -0.0272     |
| PartSop_IntervalsAugmentedAsc_Per     | 0.0250      |
| PartSop_IntervallicMean               | -0.0228     |
| PartSop_IntervalsPerfectDesc_Per      | 0.0221      |
| PartSop_IntervalsMinorDesc_Per        | -0.0221     |
| PartSop_IntervalsAugmentedDesc_Per    | 0.0215      |
| PartSop_LeapsAsc_Per                  | 0.0206      |
| PartSop_SyllabicRatio                 | 0.0191      |
| PartSop_IntervalsDiminishedAsc_Per    | 0.0180      |
| PartSop_StepwiseMotionAll_Per         | -0.0173     |
| PartSop_IntervalsMajorAll_Per         | 0.0150      |
| PartSop_IntervalsPerfectAll_Per       | 0.0136      |
| PartSop_LeapsDesc_Per                 | 0.0128      |

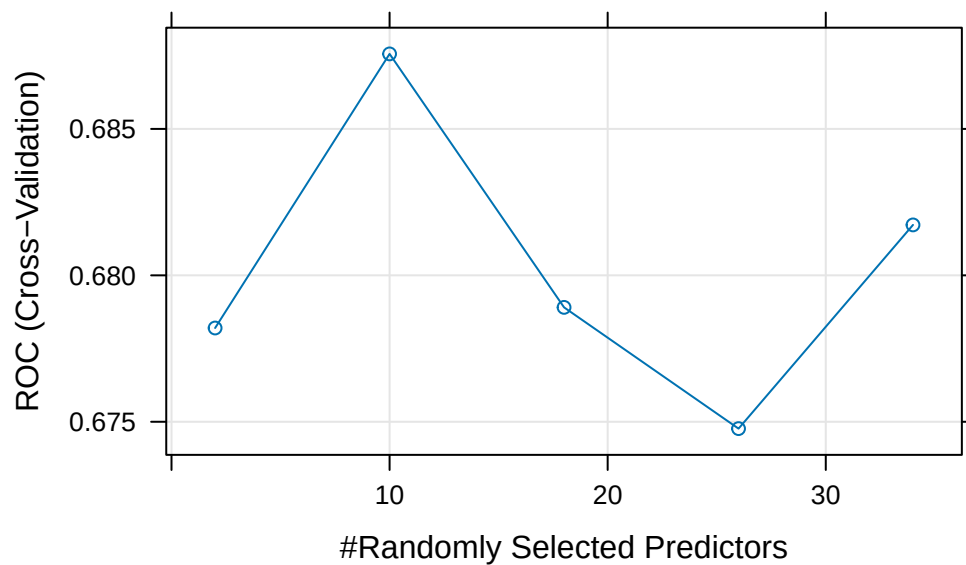
## Random forests

### Fitting

```
# Set seeds for reproducibility in caret with RF
set.seed(42)
n_folds <- 10
n_tune <- 5
rf_seeds <- c(
  lapply(seq_len(n_folds), function(i) rep(i, n_tune)),
  list(999L)
)

# Define 10-fold CV
cv_ctrl <- trainControl(method = "cv", number = 10, classProbs = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final", seeds = rf_seeds)

# Train Random Forest with tuning of mtry optimizing AUC
rf_train <- train(x = train_bake_no_y, y = train_bake$y, method = "rf",
  metric = "ROC", trControl = cv_ctrl, tuneLength = n_tune)
plot(rf_train)
```



## Predictions

```
# Predictions
y_test_hat_RF <- predict(rf_train, newdata = test_train_bake_no_y)
positive_class <- levels(train_bake$y)[2] # "Class 1" is the second factor level
p_test_hat_RF <- predict(rf_train, newdata = test_train_bake_no_y,
                          type = "prob")[, positive_class]
```

## Metrics

```
# Metrics
acc_RF <- mean(test_train_bake$y == y_test_hat_RF)
auc_RF <- auc_binary(test$y, p_test_hat_RF)
f1_RF <- f1(test$y, y_test_hat_RF)
data.frame("Accuracy" = acc_RF, "AUC" = auc_RF, "Macro_F1" = f1_RF) |>
  kable(digits = 4)
```

| Accuracy | AUC    | Macro_F1 |
|----------|--------|----------|
| 0.6559   | 0.6936 | 0.6491   |

## Variables importance

```
# Variables importance
rf_importance <- varImp(rf_train)$importance
rf_importance |>
  rownames_to_column("feature") |>
  arrange(desc(Overall)) |>
  head(30) |>
  kable(digits = 4)
```

| feature                               | Overall  |
|---------------------------------------|----------|
| PartSop_DescendingIntervallicStd      | 100.0000 |
| PartSop_VoicePresence                 | 86.6189  |
| PartSop_AbsoluteIntervallicTrimRatio  | 82.0883  |
| PartSop_IntervallicKurtosis           | 78.7317  |
| PartSop_AscendingIntervallicMean      | 76.3007  |
| PartSop_LeapsAsc_Per                  | 74.2269  |
| PartSop_HighestNoteIndex              | 74.0960  |
| PartSop_SyllabicRatio                 | 73.9419  |
| PartSop_IntervalsMinorAll_Per         | 73.4098  |
| PartSop_IntervalsPerfectDesc_Per      | 69.7022  |
| PartSop_IntervallicMean               | 68.8861  |
| PartSop_TrimmedAbsoluteIntervallicStd | 66.1748  |
| PartSop_AverageDuration               | 64.3302  |
| PartSop_IntervallicTrimDiff           | 64.0728  |
| PartSop_RhythmInt                     | 63.9892  |
| PartSop_IntervalsMinorDesc_Per        | 62.8356  |
| PartSop_LeapsDesc_Per                 | 61.5359  |
| PartSop_StepwiseMotionAll_Per         | 60.2716  |
| PartSop_IntervallicTrimRatio          | 59.7026  |
| PartSop_IntervalsPerfectAsc_Per       | 59.5378  |
| PartSop_IntervalsBeyondOctaveAsc_Per  | 59.1291  |
| PartSop_IntervalsMajorAsc_Per         | 58.9455  |
| PartSop_Ambitus                       | 58.0607  |
| PartSop_IntervalsMajorAll_Per         | 56.0912  |
| PartSop_IntervalsPerfectAll_Per       | 56.0571  |
| PartSop_DottedRhythm                  | 47.7773  |
| PartSop_IntervalsDiminishedDesc_Per   | 46.8935  |
| PartSop_LargestSemitonesAsc           | 43.2580  |
| PartSop_IntervalsAugmentedAsc_Per     | 43.1711  |
| PartSop_IntervalsDiminishedAsc_Per    | 33.5053  |

## Metrics for all models on test data

```

# Save predictions for all models
dir.create("results/y_hats", showWarnings = FALSE)
datasets_to_save <- list(
  RL = y_test_hat_RL,
  LL = y_test_hat_LL,
  RF = y_test_hat_RF,
  y_true = test$y
)
for (name in names(datasets_to_save)) {

  data_to_write <- datasets_to_save[[name]]

  # If it's a vector, convert to data frame
  if (is.vector(data_to_write)) {
    data_to_write <- data.frame(prediction = data_to_write)
  }

  # Construct filename
  file_name <- paste0("results/y_hats/", name, "_", experiment,
    "_sub_", subset_name, ".csv")

  # Write CSV
  write.csv(data_to_write, file_name, row.names = FALSE)

}

```

```

# Combine all metrics into a single table
metrics_table <- data.frame(
  Accuracy = c(acc_RL, acc_LL, acc_RF),
  AUC = c(auc_RL, auc_LL, auc_RF),
  Macro_F1 = c(f1_RL, f1_LL, f1_RF),
  Model = c("Ridge Logistic (RL)", "Lasso Logistic (LL)",
    "Random Forest (RF)")
)
metrics_table |>
  kable(digits = 4)

```

| Accuracy | AUC    | Macro_F1 | Model               |
|----------|--------|----------|---------------------|
| 0.6452   | 0.6888 | 0.6329   | Ridge Logistic (RL) |
| 0.6774   | 0.7111 | 0.6710   | Lasso Logistic (LL) |
| 0.6559   | 0.6936 | 0.6491   | Random Forest (RF)  |

## Bootstrap tests for model comparison

```
# Run bootstrapping of accuracy with predictions
B <- 10000
boot_RL <- boot_acc(test$y, y_test_hat_RL, n_boot = B, seed = 123)
boot_LL <- boot_acc(test$y, y_test_hat_LL, n_boot = B, seed = 123)
boot_RF <- boot_acc(test$y, y_test_hat_RF, n_boot = B, seed = 123)
boot <- list(RL = boot_RL, LL = boot_LL, RF = boot_RF)

# Bootstrap accuracy differences between RF and other models
diff_boot_RF_RL <- boot$RF - boot$RL
diff_boot_RF_LL <- boot$RF - boot$LL

# One-sided p-values (H1: RF better than other model)
p_RF_vs_RL <- (1 + sum(diff_boot_RF_RL <= 0)) / (B + 1)
p_RF_vs_LL <- (1 + sum(diff_boot_RF_LL <= 0)) / (B + 1)

# Two-sided 95% confidence intervals for differences
ci_diff_RL <- quantile(diff_boot_RF_RL, probs = c(0.025, 0.975))
ci_diff_LL <- quantile(diff_boot_RF_LL, probs = c(0.025, 0.975))

# Add bootstrap tests to metrics table
metrics_table <- metrics_table |>
  mutate(
    ci_diff_lo = c(ci_diff_RL[1], ci_diff_LL[1], NA),
    ci_diff_up = c(ci_diff_RL[2], ci_diff_LL[2], NA),
    pval_better = c(p_RF_vs_RL, p_RF_vs_LL, NA)
  )
metrics_table |>
  kable(digits = 4)
```

| Accuracy | AUC    | Macro_F1 | Model               | ci_diff_lo | ci_diff_up | pval_better |
|----------|--------|----------|---------------------|------------|------------|-------------|
| 0.6452   | 0.6888 | 0.6329   | Ridge Logistic (RL) | -0.0484    | 0.0699     | 0.4000      |
| 0.6774   | 0.7111 | 0.6710   | Lasso Logistic (LL) | -0.0860    | 0.0430     | 0.7711      |
| 0.6559   | 0.6936 | 0.6491   | Random Forest (RF)  | NA         | NA         | NA          |

## Comparison with majority class baseline

```
# Majority Class (MC) prediction baseline - always predict most common class
class_probs <- table(train$y) / length(train$y)
majority_class_train <- names(sort(class_probs, decreasing = TRUE))[1]
y_test_hat_MC <- factor(rep(majority_class_train, length(test$y)),
  levels = levels(test$y))
```

```

# Metrics for majority class baseline
acc_MC <- mean(test$y == y_test_hat_MC)
p_test_hat_MC <- rep(class_probs[majority_class_train], length(test$y))
auc_MC <- auc_binary(test$y, p_test_hat_MC)
f1_MC <- f1(test$y, y_test_hat_MC)

# Bootstrap accuracy for majority class baseline
boot_MC <- boot_acc(test$y, y_test_hat_MC, n_boot = B, seed = 123)

# Bootstrap accuracy differences between majority class baseline and models
diff_boot_RL_MC <- boot_RL - boot_MC
diff_boot_LL_MC <- boot_LL - boot_MC
diff_boot_RF_MC <- boot_RF - boot_MC

# One-sided p-values (H1: model better than majority class baseline)
p_RL_vs_MC <- (1 + sum(diff_boot_RL_MC <= 0)) / (B + 1)
p_LL_vs_MC <- (1 + sum(diff_boot_LL_MC <= 0)) / (B + 1)
p_RF_vs_MC <- (1 + sum(diff_boot_RF_MC <= 0)) / (B + 1)

# Two-sided 95% confidence intervals for differences
ci_diff_RL_MC <- quantile(diff_boot_RL_MC, probs = c(0.025, 0.975))
ci_diff_LL_MC <- quantile(diff_boot_LL_MC, probs = c(0.025, 0.975))
ci_diff_RF_MC <- quantile(diff_boot_RF_MC, probs = c(0.025, 0.975))

# Combine all metrics into a single table
metrics_table_MC <- data.frame(
  Accuracy = c(acc_RL, acc_LL, acc_RF, acc_MC),
  AUC = c(auc_RL, auc_LL, auc_RF, auc_MC),
  Macro_F1 = c(f1_RL, f1_LL, f1_RF, f1_MC),
  ci_diff_lo = c(ci_diff_RL_MC[1], ci_diff_LL_MC[1], ci_diff_RF_MC[1], NA),
  ci_diff_up = c(ci_diff_RL_MC[2], ci_diff_LL_MC[2], ci_diff_RF_MC[2], NA),
  pval_better = c(p_RL_vs_MC, p_LL_vs_MC, p_RF_vs_MC, NA),
  Model = c("Ridge Logistic (RL)", "Lasso Logistic (LL)",
            "Random Forest (RF)", "Majority Class (MC)")
)
metrics_table_MC |>
  kable(digits = 4)

```

| Accuracy | AUC    | Macro_F1 | ci_diff_lo | ci_diff_up | pval_better | Model               |
|----------|--------|----------|------------|------------|-------------|---------------------|
| 0.6452   | 0.6888 | 0.6329   | 0.0108     | 0.1828     | 0.0152      | Ridge Logistic (RL) |
| 0.6774   | 0.7111 | 0.6710   | 0.0376     | 0.2204     | 0.0023      | Lasso Logistic (LL) |
| 0.6559   | 0.6936 | 0.6491   | 0.0161     | 0.1989     | 0.0110      | Random Forest (RF)  |
| 0.5484   | 0.5000 | 0.3542   | NA         | NA         | NA          | Majority Class (MC) |

```
# In a combined table
metrics_table_combined <- metrics_table_MC |>
  mutate(ci_diff_lo_MC = ci_diff_lo, ci_diff_up_MC = ci_diff_up,
         pval_better_MC = pval_better) |>
  mutate(ci_diff_lo_RF = c(metrics_table$ci_diff_lo, NA),
         ci_diff_up_RF = c(metrics_table$ci_diff_up, NA),
         pval_better_RF = c(metrics_table$pval_better, NA)) |>
  select(Model, Accuracy, AUC, Macro_F1,
         ci_diff_lo_RF, ci_diff_up_RF, pval_better_RF,
         ci_diff_lo_MC, ci_diff_up_MC, pval_better_MC)
metrics_table_combined |>
  kable(digits = 4)
```

| Model                  | Accu-<br>racy | AUC    | Macro_F1 | ci_diff_lo_RF | ci_diff_up_RF | pval_bet-<br>ter_RF | ci_diff_lo_MC | ci_diff_up_MC | pval_bet-<br>ter_MC |
|------------------------|---------------|--------|----------|---------------|---------------|---------------------|---------------|---------------|---------------------|
| Ridge                  | 0.6452        | 0.6888 | 0.6329   | -0.0484       | 0.0699        | 0.4000              | 0.0108        | 0.1828        | 0.0152              |
| Logistic<br>(RL)       |               |        |          |               |               |                     |               |               |                     |
| Lasso                  | 0.6774        | 0.7111 | 0.6710   | -0.0860       | 0.0430        | 0.7711              | 0.0376        | 0.2204        | 0.0023              |
| Logistic<br>(LL)       |               |        |          |               |               |                     |               |               |                     |
| Random<br>Forest (RF)  | 0.6559        | 0.6936 | 0.6491   | NA            | NA            | NA                  | 0.0161        | 0.1989        | 0.0110              |
| Majority<br>Class (MC) | 0.5484        | 0.5000 | 0.3542   | NA            | NA            | NA                  | NA            | NA            | NA                  |

```
# Save table of all metrics + bootstrap tests
dir.create("results/experiments", showWarnings = FALSE)
write.csv(metrics_table_combined, paste0("results/experiments/all_results_",
                                         "Experiment_", experiment,
                                         "_sub_", subset_name, ".csv"),
          row.names = FALSE)
```

```
# Print LaTeX table
latex_table <- metrics_table_combined |>
  mutate(ci_RF = paste0("$(", sprintf("%.4f", ci_diff_lo_RF), ", ",
                                     sprintf("%.4f", ci_diff_up_RF), ")$"),
         ci_MC = paste0("$(", sprintf("%.4f", ci_diff_lo_MC), ", ",
                                     sprintf("%.4f", ci_diff_up_MC), ")$"),
         p_RF = sprintf("%.4f", pval_better_RF),
         p_MC = sprintf("%.4f", pval_better_MC)) |>
  select(Model, Accuracy, AUC, Macro_F1, ci_RF, p_RF, ci_MC, p_MC)
latex_table |>
  kable(digits = 4)
```

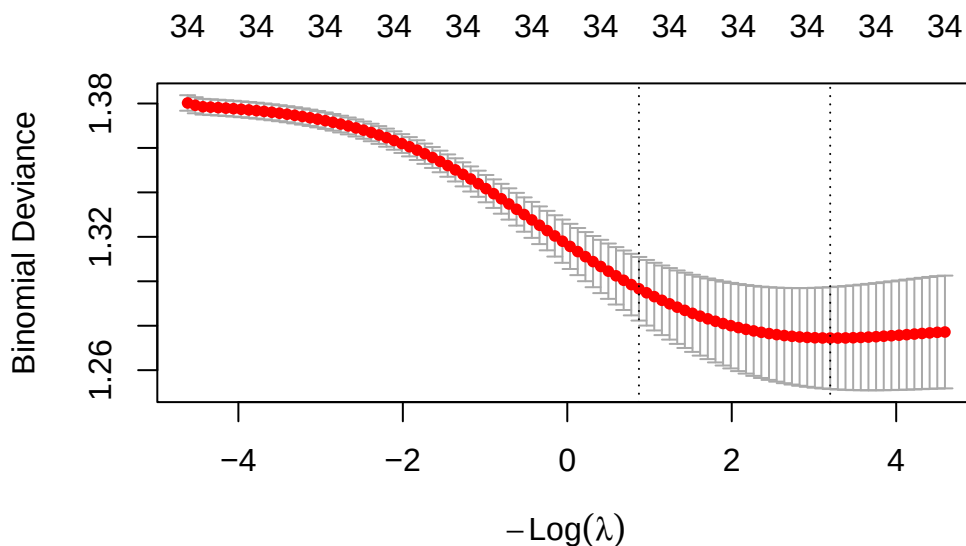
| Model               | Accuracy | AUC    | Macro_F1 | ci_RF             | p_RF   | ci_MC            | p_MC   |
|---------------------|----------|--------|----------|-------------------|--------|------------------|--------|
| Ridge Logistic (RL) | 0.6452   | 0.6888 | 0.6329   | (-0.0484, 0.0699) | 0.4000 | (0.0108, 0.1828) | 0.0152 |
| Lasso Logistic (LL) | 0.6774   | 0.7111 | 0.6710   | (-0.0860, 0.0430) | 0.7711 | (0.0376, 0.2204) | 0.0023 |
| Random Forest (RF)  | 0.6559   | 0.6936 | 0.6491   | (NA, NA)          | NA     | (0.0161, 0.1989) | 0.0110 |
| Majority Class (MC) | 0.5484   | 0.5000 | 0.3542   | (NA, NA)          | NA     | (NA, NA)         | NA     |

```
latex_table |>
  kable(digits = 4, format = "latex", booktabs = TRUE, escape = FALSE) |>
  print()
```

```
\begin{tabular}{lrrrrlllll}
\toprule
Model & Accuracy & AUC & Macro_F1 & ci_RF & p_RF & ci_MC & p_MC\\
\midrule
Ridge Logistic (RL) & 0.6452 & 0.6888 & 0.6329 &  $\$(-0.0484, 0.0699)\$$  & 0.4000 &  $\$(0.0108, 0.1828)\$$  & 0.0152\\
Lasso Logistic (LL) & 0.6774 & 0.7111 & 0.6710 &  $\$(-0.0860, 0.0430)\$$  & 0.7711 &  $\$(0.0376, 0.2204)\$$  & 0.0023\\
Random Forest (RF) & 0.6559 & 0.6936 & 0.6491 &  $\$(NA, NA)\$$  & NA &  $\$(0.0161, 0.1989)\$$  & 0.0110\\
Majority Class (MC) & 0.5484 & 0.5000 & 0.3542 &  $\$(NA, NA)\$$  & NA &  $\$(NA, NA)\$$  & NA\\
\bottomrule
\end{tabular}
```

## Ridge logistic for inference and final model on all data (train + test)

```
# Fit ridge
set.seed(42)
all_bake_no_y <- all_bake |>
  select(-y) |>
  as.matrix()
cv0_all <- cv.glmnet(x = all_bake_no_y, y = all_bake$y, family = "binomial",
  alpha = 0, nfolds = 10)
plot(cv0_all)
```



```
cv0_all
```

Call: `cv.glmnet(x = all_bake_no_y, y = all_bake$y, nfolds = 10, family = "binomial", alpha = 0)`



Measure: Binomial Deviance

|     | Lambda | Index | Measure | SE      | Nonzero |
|-----|--------|-------|---------|---------|---------|
| min | 0.0409 | 85    | 1.274   | 0.02293 | 34      |
| 1se | 0.4183 | 60    | 1.297   | 0.01431 | 34      |

```
# Confidence intervals for all coefficients
ci_df <- logistic_ridge_ci(X = all_bake_no_y, y = all_bake$y,
                          beta_hat = coef(cv0_all, s = cv0_all$lambda.1se),
                          lambda = nrow(all_bake) * cv0_all$lambda.1se / 2)

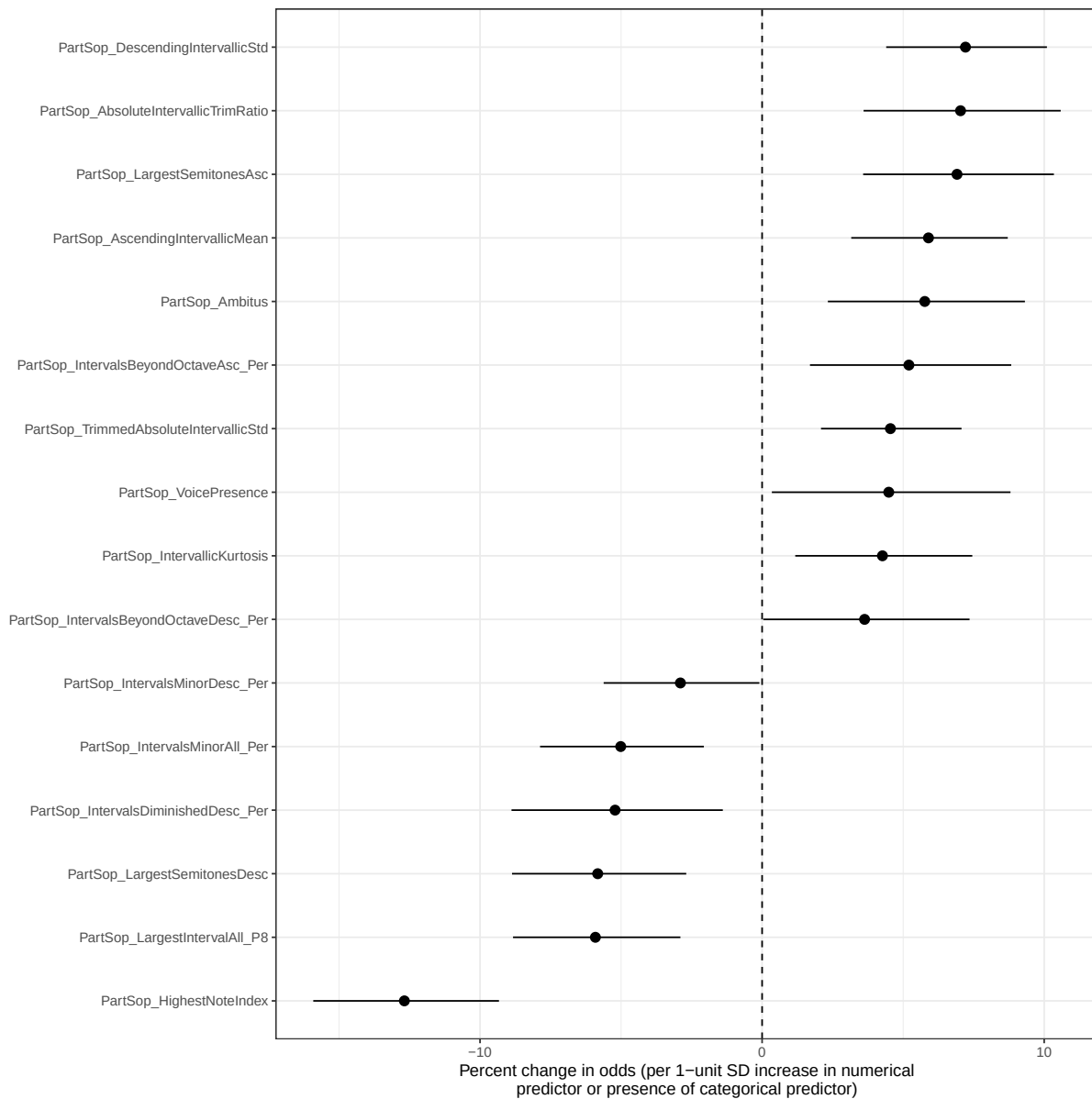
# Filter significant coefficients and order by effect size
ci_df <- ci_df |>
  filter(sign(ci_low * ci_up) > 0) |>
  mutate(
    pct_change = (exp(estimate) - 1) * 100,
    pct_low = (exp(ci_low) - 1) * 100,
    pct_up = (exp(ci_up) - 1) * 100
  ) |>
  mutate(
    term = factor(term, levels = term[order(pct_change, decreasing = FALSE)])
  )
ci_df |>
  select(term, pct_change, pct_low, pct_up, p_value) |>
  arrange(desc(pct_change)) |>
  kable(digits = 2)
```

| term                                  | pct_change | pct_low | pct_up | p_value |
|---------------------------------------|------------|---------|--------|---------|
| PartSop_DescendingIntervallicStd      | 7.21       | 4.40    | 10.10  | 0.00    |
| PartSop_AbsoluteIntervallicTrimRatio  | 7.03       | 3.60    | 10.59  | 0.00    |
| PartSop_LargestSemitonesAsc           | 6.91       | 3.58    | 10.34  | 0.00    |
| PartSop_AscendingIntervallicMean      | 5.90       | 3.16    | 8.71   | 0.00    |
| PartSop_Ambitus                       | 5.77       | 2.33    | 9.32   | 0.00    |
| PartSop_IntervalsBeyondOctaveAsc_Per  | 5.20       | 1.70    | 8.83   | 0.00    |
| PartSop_TrimmedAbsoluteIntervallicStd | 4.55       | 2.09    | 7.07   | 0.00    |
| PartSop_VoicePresence                 | 4.49       | 0.35    | 8.81   | 0.03    |
| PartSop_IntervallicKurtosis           | 4.27       | 1.17    | 7.45   | 0.01    |
| PartSop_IntervalsBeyondOctaveDesc_Per | 3.63       | 0.04    | 7.35   | 0.05    |
| PartSop_IntervalsMinorDesc_Per        | -2.90      | -5.62   | -0.10  | 0.04    |
| PartSop_IntervalsMinorAll_Per         | -5.01      | -7.87   | -2.06  | 0.00    |
| PartSop_IntervalsDiminishedDesc_Per   | -5.21      | -8.88   | -1.39  | 0.01    |
| PartSop_LargestSemitonesDesc          | -5.83      | -8.87   | -2.69  | 0.00    |
| PartSop_LargestIntervalAll_P8         | -5.91      | -8.83   | -2.90  | 0.00    |
| PartSop_HighestNoteIndex              | -12.68     | -15.91  | -9.33  | 0.00    |

```

# Plot of coefficients in odds
plot_RL <- ggplot(ci_df, aes(x = term, y = pct_change,
                             ymin = pct_low, ymax = pct_up)) +
  geom_pointrange() +
  geom_hline(yintercept = 0, linetype = 2) +
  coord_flip() +
  labs(
    x = NULL,
    y = "Percent change in odds (per 1-unit SD increase in numerical\npredictor or presence of category)"
  ) +
  theme_bw()
plot_RL

```



```
# Save plot
plot_name <- paste0("plots/plot_rl", "_", experiment,
                    "_sub_", subset_name, ".png")
ggsave(filename = plot_name, plot = plot_RL, width = 8, height = 6, dpi = 300)

# Metrics on all data available, just for reference
y_all_hat_RL <- predict(cv0_all, newx = all_bake_no_y, s = "lambda.1se",
                        type = "class")
p_all_hat_RL <- predict(cv0_all, newx = all_bake_no_y, s = "lambda.1se",
                        type = "response")
```

```

acc_all_RL <- mean(all_bake$y == y_all_hat_RL)
auc_all_RL <- auc_binary(all_bake$y, p_all_hat_RL)
f1_all_RL <- f1(all_bake$y, y_all_hat_RL)
glmnet_summary <- data.frame("Accuracy" = acc_all_RL, "AUC" = auc_all_RL,
                             "Macro_F1" = f1_all_RL,
                             "Model" = "Ridge Logistic (RL)")

glmnet_summary |>
  kable(digits = 4)

```

| Accuracy | AUC    | Macro_F1 | Model               |
|----------|--------|----------|---------------------|
| 0.6322   | 0.6926 | 0.6229   | Ridge Logistic (RL) |

```

# Save results
dir.create("results/final_model", showWarnings = FALSE)
write.csv(glmnet_summary, paste0("results/final_model/RL_final_model_results",
                                "_Experiment_", experiment, "_sub_",
                                subset_name, ".csv"),
          row.names = FALSE)

```